



International University of Business Agriculture and Technology

Assignment

Course Code: CSE 1383

Course Name: Structured Programming

Submitted to:

D.M Arif Afsar

Senior Lecturer

Department of Computer Science and Engineering

Submitted by:

Md. Irfan Khan

ID:25303040

Section: D

Program: BCSE

Contents

No.	Topic Name	Page
1	Introduction	3
2	Problem Analysis	5
3	System Design	8
4	Implementation Details	15
5	Test Cases	17
6	Discussion	20
7	Conclusion	21

Introduction

Every semester, schools and colleges face a big challenge: organizing exams. They need to manage physical rooms, count the seats, and match them with student identification numbers. This process is called examination hall management. Managing exam halls is not just about giving rooms to students. It is about creating a quiet, organized, and fair place where students can focus on their tests. However, doing this work by hand (manually) causes many problems and delays.

Examination Hall Management Challenges

When teachers and staff manage seating arrangements using paper lists or basic spreadsheets, they face these common problems:

- **High Risk of Mistakes:** Writing or typing hundreds of student IDs into seat lists takes a long time. It is very easy to make a mistake, like giving the same seat to two students or forgetting a student completely. This confuses right before the exam starts.
- **Bad Use of Space:** Without a clear, real-time view of all rooms, some exam halls become too crowded while others stay mostly empty. This wastes school space and makes it difficult to assign teachers to watch the students.
- **Delays on Exam Day:** On the day of the exam, students often struggle to find where they need to sit. If a student forgets their room number, teachers must search through long paper lists. This makes students stressed and delays the start of the test.
- **Difficult to Count Data:** Calculating how many seats are full or empty by hand takes hours of extra paperwork. Without fast numbers, managers cannot make quick decisions during exam week.

The Need for Automation

To fix these problems, schools need an automated computer system. A simple program—like the one created in this project—makes the whole process much easier because of these benefits:

- **Fast Allocation Without Mistakes:** The computer program can give a specific hall and seat number to every student ID in less than a second. This completely stops duplicate seating or lost names.
- **Warning for Crowded Rooms:** The system automatically calculates the percentage of full seats. If a hall is more than 90% full, the program gives a warning so staff can fix the crowding before it becomes a problem.
- **Saving School Resources:** The program can easily find rooms that are mostly empty. For example, if a hall is less than 25% full, the system suggests moving those students to another room. This saves electricity and requires fewer teachers to watch the exam.
- **Quick Student Search:** Finding information becomes very simple. If a student loses their seat number, teachers can just type the student's ID into the system to find their exact room and seat immediately.
- **All-in-One Reports:** Instead of creating many separate lists, the program can print one complete report. This report shows the seating pattern, empty seats, crowded room warnings, and helpful suggestions in one clear view.

Problem Analysis

To understand how the Examination Hall Management program works, we need to look at its four main parts: what goes into the program (Input), what comes out (Output), the rules it must follow (Constraints), and how it saves data (Array Structure).

Input

The program asks the user to type four main pieces of information using the keyboard:

- **Number of Halls:** How many rooms are used for the exam.
- **Number of Seats per Hall:** How many chairs are available in each room.
- **Number of Students:** The total number of students who need a seat.
- **Student ID:** A specific identification number used to search for a student's seat at the end of the program.

Output

The program processes the inputs and prints a clear report on the screen. The outputs include:

- **Seating Pattern:** A visual map of each hall. It shows the Student ID (starting from 25303001) if a seat is full, or an X if the seat is empty.
- **Empty Seat Finder:** A list that shows the exact hall number and seat position for every empty seat.

- **Overloaded Hall Warnings:** A message that warns teachers if a hall is too crowded (90% or more full).
- **Optimization Suggestions:** A recommendation message. If a hall is less than 25% full, the program tells the user to merge it with another room to save space.
- **Search Results:** When a user types a Student ID, the program outputs the exact hall and seat number for that student. If the ID does not exist, it prints "Not Found..".

Constraints

Constraints are the strict limits and rules that the program must follow. If the user inputs a wrong number, the program will ask them to enter it again. The rules are:

- **Hall Limits:** The number of halls must be between 1 and 3.
- **Seat Limits:** The number of seats per hall must be between 1 and 10.
- **Student Limits:** The number of students cannot be 0 or less, and it cannot be more than the maximum seats available (Halls *Seats).
- **Crowding Warning Limit:** The warning for an overloaded hall only triggers if the full seats reach 90% or more.
- **Merging Limit:** The suggestion to combine halls only triggers if the full seats are less than 25%.

Array Structure

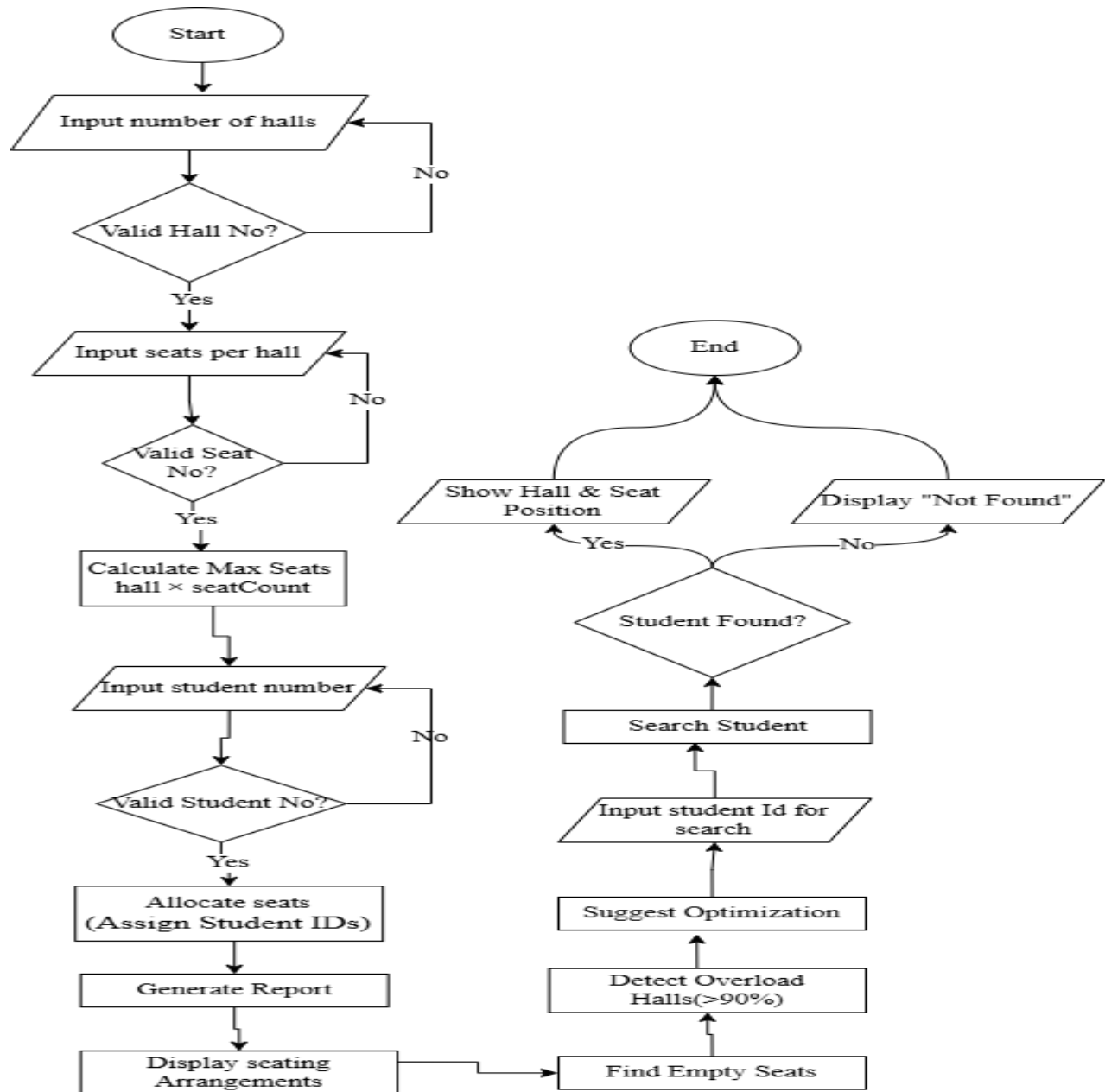
The program uses a 2D (two-dimensional) integer array called `hallArray[3][10]` to save all the seating data. You can think of this array as a grid or a table:

- **Rows represent the Exam Halls:** There is a maximum of 3 rows (Row 0, Row 1, and Row 2).
- **Columns represent the Seats:** There is a maximum of 10 columns in each row (Columns 0 to 9) to represent the 10 seats.
- **Values inside the Grid:** If a seat is taken, the grid stores the Student ID number .
 - If a seat is empty, the grid stores the number 0.

System Design

System design shows how the program is structured and how data moves from one step to another. It helps us understand the parts of the program and how they work together.

Flowchart:



Function Decomposition: Exam Hall Seat Management System:

1. main()

This is the starting point of the program. It controls everything and connects all other functions together.

What it does:

- Takes input for number of halls, seats per hall, and number of students
- Checks whether the inputs are valid
- Calls functions to allocate seats, generate reports, and search students

2. allocateSeats()

This function is responsible for assigning seats to students automatically.

What it does:

- Starts student IDs from 25303001
- Fills the hall seating system one by one
- Assigns IDs to seats until all students are placed
- Marks remaining seats as empty (0)

3. hallOccupancy()

This function calculates how full each exam hall is.

What it does:

- Counts how many seats are occupied in each hall
- Finds how many seats are still empty

- Calculates occupancy percentage
- Displays the result for each hall

4. detectOverloaded()

This function checks if any hall is too crowded.

What it does:

- Calculates occupancy percentage for every hall
- If a hall is 90% or more full, it shows a warning message

5. findEmptySeat()

This function helps to locate all empty seats.

What it does:

- Checks each seat in every hall
- Identifies seats that are not assigned (value is 0)
- Prints their hall number and position

6. searchStudent()

This function is used to find a specific student in the seating system.

What it does:

- Searches all halls and seats
- If the student ID is found, it shows the hall and seat number
- If not found, it displays a “Not Found” message

7. displaySeatArrangements()

This function prints the full seating layout.

What it does:

- Shows student IDs for occupied seats
- Shows “X” for empty seats
- Displays hall-by-hall seating structure

8. suggestOptimization()

This function gives suggestions to improve seat usage.

What it does:

- Checks occupancy of each hall
- If a hall is less than 25% full, suggests merging it with another hall
- Otherwise, it says no optimization is needed

9. reportGenerate()

This function creates a complete summary report.

What it does:

- Shows total halls and students
- Calls other functions to display:
 - Seating arrangement
 - Empty seats
 - Overloaded halls
 - Optimization suggestions

Algorithm:**Step 1: Start****Step 2: Input and Validation**

1. Input number of halls (hallCount).
2. If $\text{hallCount} < 1$ or $\text{hallCount} > 3$, ask again.
3. Input seats per hall (seatCount).
4. If $\text{seatCount} < 1$ or $\text{seatCount} > 10$, ask again.
5. Calculate maximum seats:
 - $\text{maxSeat} = \text{hallCount} \times \text{seatCount}$
6. Input number of students (studentCount).
7. If $\text{studentCount} < 1$ or $\text{studentCount} > \text{maxSeat}$, ask again.

Step 3: Automatic Seat Allocation

1. Set first student ID = 25303001.
2. For each hall:
 - For each seat:
 - If students remain unassigned:
 - Assign current student ID to seat.
 - Increment student ID.
 - Otherwise:
 - Mark seat as empty (0).

Step 4: Hall Occupancy Analysis

1. For each hall:
 - Count occupied seats.
 - Calculate vacant seats.
 - Calculate occupancy percentage.
2. Display occupancy percentage and vacant seats.

Step 5: Detect Overloaded Halls

1. For each hall:
 - Calculate occupancy percentage.
 - If occupancy percentage $\geq 90\%$:
 - Display overload warning.

Step 6: Find Empty Seats

1. For each hall:
 - Check every seat.
 - If seat value is 0:
 - Display hall number and seat position.

Step 7: Display Seating Arrangement

1. For each hall:
 - Display student IDs for occupied seats.
 - Display "X" for empty seats.

Step 8: Optimization Suggestion

1. For each hall:

- Calculate occupancy percentage.
- If occupancy percentage $< 25\%$:
 - Suggest merging the hall with another hall.
- 2. Otherwise:
 - Display "No optimization suggestion".

Step 9: Generate Overall Report

1. Display total halls.
2. Display total students.
3. Display seating arrangement.
4. Display empty seats.
5. Display overloaded halls.
6. Display optimization suggestions.

Step 10: Student Search Module

1. Input a student ID.
2. Search all halls and seats.
3. If found:
 - Display hall number and seat position.
4. Otherwise:
 - Display "Not Found".

Step 11: End

Implementation Details

1.Seat Allocation Logic

- The program uses a 2D array to store hall seats
- Each row = one hall, each column = one seat
- The function `allocateSeats()` assigns student IDs starting from 25303001
- Seats are filled one by one from first hall to last hall
- If seats are still empty, they are marked as 0
- This helps to separate filled and empty seats easily

2.Searching Logic

- The function `searchStudent()` is used for searching a student ID
- It checks all halls and all seats one by one
- This method is called linear search
- If the ID is found:
 - It shows hall number and seat number
- If the ID is not found:
 - It shows “Not Found” message
- This is a simple and easy search method

3.Occupancy Analysis

- Done using `hallOccupancy()` and `detectOverloaded()` functions
- First, the program counts occupied seats in each hall

- Then it calculates empty seats
- Occupancy percentage is calculated using:
 - $\text{Occupancy \%} = (\text{Occupied seats} \div \text{Total seats}) \times 100$
- If occupancy is 90% or more:
 - The hall is marked as overloaded
 - A warning message is shown
- This helps to understand hall usage clearly

Test Cases

Test Case 1:

Input:

Number of halls: 2

Seats per hall: 5

Number of students: 8

Student ID to search: 25303004

Output:

...Overall report...

Total hall: 2

Total student: 8

Seat Arrangements...

Hall 1:

25303001 25303002 25303003 25303004 25303005

Hall 2:

25303006 25303007 25303008 X X

Empty seats...

Hall 2: Empty seat position 4

Hall 2: Empty seat position 5

Overloaded hall...

(No hall exceeds 90%)

Optimization suggestion...

(No suggestion if hall occupancy > 25%)

Search Result:

Hall 1: ID:25303004 seat position is 4

Test Case 2:

Input:

Number of halls: 3

Seats per hall: 10

Number of students: 30

Student ID to search: 25303025

Output:

...Overall report...

Total hall: 3

Total student: 30

Seat Arrangements...

Hall 1: 25303001 25303002 ... 25303010

Hall 2: 25303011 25303012 ... 25303020

Hall 3: 25303021 25303022 ... 25303030

Empty seats... (No empty seats)

Overloaded hall...

Warning: Hall-1 exceeds 90 percent

Warning: Hall-2 exceeds 90 percent

Warning: Hall-3 exceeds 90 percent

Optimization suggestion...

(No suggestion)

Search Result:

Hall 3: ID:25303025 seat position is 5

Discussion

Limitations

- The program uses a fixed-size array (maximum 3 halls and 10 seats per hall), so it is not flexible for larger systems
- The search method is linear search, so it may be slow if the data becomes very large
- It does not store data permanently (no file or database), so all data is lost after program ends
- There is no option to update or delete a student after seat allocation
- Input validation is basic and may not handle all incorrect inputs

Improvements

- The program can be improved by using dynamic memory allocation to support more halls and seats
- A faster search method (like binary search or hash table) can be used for large data
- File handling or database can be added to store student data permanently
- Features like updating or removing a student can be added
- A better user interface (menu-driven system) can make it easier to use
- More advanced error handling can improve input validation

Conclusion

This project helped me understand how a real-life problem like exam hall seat management can be solved using C programming. I learned how to use a 2D array to store and manage data in a simple way. I also understood how to divide a large problem into smaller functions such as seat allocation, searching, and occupancy analysis. Through this project, I practiced important programming concepts like loops, conditions, and function calling. Overall, this work improved my problem-solving skills and helped me gain a better understanding of structured programming in C.